

Bano Qabil × Alibaba Cloud AI Hackathon 2026 — Complete Team Blueprint

Prepared for a 5-member beginner team. Domain choice, problem, architecture, roles, and demo plan included.

STEP 1 — Choosing the Domain

Domain	Real problem opportunities	Social impact	Innovation potential	AI suitability	Beginner difficulty	Data/API availability	Prototype-ability	Alibaba Cloud fit	Demo potential
Smart Agriculture	Crop disease, pricing, water use — huge, under-digitized	Very high (farmers = ~37% of Pakistan's workforce)	High if paired with voice/local language	High (vision + NLP)	Medium	Medium (some open crop-disease image sets)	High — a photo-in, advice-out demo is easy to fake live	Strong (Qwen-VL + Qwen-Plus)	Very high — visual, emotional
Financial Inclusion	Loan literacy, budgeting for informal workers	High	Medium — many fintech chatbots already exist	Medium	Medium-High (needs real financial data, regulation-sensitive)	Low (real financial data is hard to get, sensitive)	Medium	Medium	Medium
Urdu & Regional Language Tech	Illiteracy, no Urdu-first tools, script/dialect gaps	High	High as an <i>enabler</i> domain	Very high	Medium	Good (open Urdu corpora, Qwen has decent Urdu)	High	Very strong (Qwen ASR/TTS/LLM)	High
Healthcare	Doctor shortage, diagnosis delay	Very high	Medium — health-chatbots are the most copied category	High	Medium-High (medical accuracy stakes, liability worries)	Medium	Medium (judges scrutinize medical claims hard)	Strong	High but risky (judges ask "is this safe?")
Education	Access to tutoring, exam prep	High	Low-Medium — "AI tutor" is the most repeated hackathon idea worldwide	High	Low	High	High	Strong	Medium
Open Innovation	Anything	Depends	Depends	Depends	Depends	Depends	Depends	Depends	Depends

Recommendation: Smart Agriculture, with Urdu voice/language technology built in as the core interface layer (not bolted on).

Why: Agriculture gives you a problem judges can viscerally understand in 10 seconds, a clear before/after story, and Alibaba Cloud's multimodal Qwen models (image understanding + Urdu-capable LLM + speech) map almost perfectly onto "farmer sends a photo and a voice note, gets an answer in Urdu." Pure "Urdu Language Tech" as a standalone domain risks becoming an abstract translation/TTS demo with no visible real-world stakes. Healthcare and Education are the most oversaturated categories at almost every hackathon, including this one, per the organizers' own goal of "practical AI solutions addressing real-world challenges." <cite index="21-1,21-2">The Alibaba Cloud AI Hackathon 2026, run by Bano Qabil and Alibaba Cloud, is a nationwide competition meant to accelerate AI innovation and give participants practical training, mentorship, and access to cloud technologies to build real-world solutions.</cite>

STEP 2 — The Real Problem

Who is suffering? Small and mid-scale crop farmers across Punjab and Sindh — most working 1–10 acres, many with limited literacy, limited smartphone fluency, and no Urdu-language digital tools built for them.

What exactly is their problem? When a crop shows disease symptoms (spots, wilting, discoloration, pest damage), the farmer usually cannot identify it early enough. Misidentification leads to wrong or excessive pesticide use, wasted money, and yield loss.

How do they currently solve it? They ask a local pesticide-shop dealer (who is incentivized to sell more product, not to diagnose correctly), rely on word-of-mouth, or wait for a government agriculture extension officer — a role that is chronically understaffed relative to the number of farms.

Why does the current solution fail? The dealer has a financial conflict of interest. Extension officers are too few to visit each farm quickly. Existing agri-apps are mostly English-first, text-heavy, or require typing — a real barrier for farmers who are more comfortable speaking than typing, especially in Urdu/Punjabi.

What happens because of this problem? Crop loss, excess/incorrect pesticide spend (a direct cost and an environmental/health issue), and lower yields — which compounds into food security and rural income pressure. Pakistan's agriculture sector accounts for roughly a fifth of GDP and about a third of the labor force, so small efficiency gains at the farm level scale nationally.

Why is this worth solving? It's not a hypothetical AI use case — voice-based Urdu agri-advisory is already being pitched commercially by regional voice-AI vendors as one of their flagship examples, which confirms real market demand: <cite index="13-1">one Pakistani-language voice AI company explicitly markets a scenario where "an AI agent will inform you about a disease in your crop and offer a free expert visit."</cite> That confirms the problem is real and commercially recognized — but as you'll see in Step 3, nobody has packaged it as a free, farmer-facing, photo+voice diagnostic prototype the way we plan to.

STEP 3 — Innovation Check

A. What already exists?

- Global "plant disease detector" apps (mostly English/Chinese-first, trained on PlantVillage-style datasets, no voice interface, no local advisory context).
- Urdu text-to-speech and speech-to-text vendors exist as pure *infrastructure* — <cite index="18-1,18-2">Speechmatics offers Urdu speech-to-text transcription across Modern Standard Urdu and regional dialects</cite>, and <cite index="13-1">Uplift AI builds Urdu/Pakistani-language voice models</cite> — but these are developer tools/call-center platforms, not farmer-facing diagnostic products.
- Enterprise Urdu voice-bot platforms exist for e-commerce/banking/telecom customer support, <cite index="12-1,12-2">explicitly serving industries like COD verification, banking, and telecom in Urdu across Pakistan and India</cite> — again, general-purpose infrastructure, not agriculture-specific diagnosis.

- PlantVillage/Kaggle-style disease classifiers exist as research projects and student GitHub repos, almost always image-in/label-out, English UI, no advisory generation, no voice.

B. What is missing from existing solutions?

1. No free, end-to-end tool that combines *photo diagnosis + spoken Urdu explanation + a locally-relevant, low-cost treatment plan* in one flow.
2. No tool designed around **low literacy** — most agri-apps assume the farmer can read and type.
3. No tool that gives a **confidence-aware answer** ("I'm fairly sure this is X, but if you see Y symptom too, get a second opinion") instead of a single overconfident label.
4. No tool that factors in **what's actually available/affordable locally** (e.g., recommends a treatment a farmer can realistically buy nearby, not a generic global answer).

C. How can our project be meaningfully different? We are not inventing crop-disease AI or Urdu TTS from scratch — those exist. Our innovation is the **combination + design for the actual user**: voice-in/voice-out in Urdu, photo-based visual diagnosis, and a low-literacy-first UI (large icons, minimal text, spoken confirmation of every step), running affordably on Alibaba Cloud's multimodal Qwen stack.

D. The exact innovation to lead with: "*Confidence-transparent, voice-first crop advisory*" — the app always tells the farmer not just *what* it thinks the problem is, but *how sure it is*, and offers a fallback ("ask a human expert") when confidence is low. This is the feature judges will remember, because it directly answers the toughest Q&A question ("what happens when your AI is wrong?") in the product design itself.

Innovation Score: 7.5/10 — honest, defensible score. It's a strong recombination of proven components for an underserved user, not a never-seen-before invention.

STEP 4 — The Final Project

Project name: FasalDoc (فصل ڈاکٹر — "Crop Doctor") **Tagline:** "*Apni Fasal Ki Awaaz Mein Baat Karein*" — "Talk to your crop, in your own voice." **Domain:** Smart Agriculture (with Urdu voice tech as the core interface)

One-sentence pitch: FasalDoc lets any farmer take a photo of a sick crop, ask a question by voice in Urdu, and get a spoken, honest, locally-relevant diagnosis and treatment plan — no typing, no reading required.

30-second pitch: Millions of small farmers in Pakistan lose money every season because they can't quickly and reliably identify what's wrong with their crops. They rely on pesticide dealers who profit from overselling. FasalDoc fixes this with a phone app: snap a photo of the affected plant, speak your question in Urdu, and get back a spoken answer — what the problem likely is, how confident the AI is, and what to do next, using treatments that are actually available locally. If the AI isn't sure, it says so and suggests contacting a real expert instead of guessing.

2-minute explanation: [Combine problem from Step 2 with solution: photo → Qwen-VL vision analysis → structured diagnosis → Qwen-Plus generates a farmer-friendly Urdu explanation → Alibaba Cloud speech synthesis reads it aloud → farmer can ask a follow-up by voice, transcribed by speech recognition, and the system responds conversationally, keeping a running context of that farmer's crop and past questions.]

Target users: Small/mid-scale crop farmers in Pakistan, particularly those less comfortable with reading/typing, and their extended families/agents who may operate the phone on their behalf.

Current pain point: No trustworthy, free, fast, voice-based diagnostic channel; dependence on conflicted local dealers.

Existing solutions & limitations: Generic plant-disease apps (English-first, text-only, single-label overconfident outputs); Urdu voice-bot infrastructure vendors (not agriculture-specific, not free/farmer-facing); extension officers (too few, too slow).

Our solution: A voice-and-photo-first mobile-web app built on Alibaba Cloud's Qwen multimodal stack, with confidence-transparent answers and locally-grounded advice.

Why AI is necessary: Visual disease pattern-matching at this speed and scale is not something a rule-based app can do; natural Urdu conversation (not rigid menus) requires an LLM; understanding an unstructured photo requires a vision model.

Why our project is different: The combination of (1) voice-first Urdu UX built for low literacy, (2) confidence-transparent answers, and (3) locally-available treatment recommendations, rather than a generic global answer.

Expected social impact: Reduced crop loss, reduced wasted/harmful pesticide spend, and a free "second opinion" channel that doesn't depend on a conflicted seller.

Why judges should care: It is demoable in under 90 seconds, solves a problem that is emotionally and economically real for Pakistan specifically, is honest about AI limitations (which most hackathon projects dodge), and has a clear, explainable Alibaba Cloud integration story.

STEP 5 — User Journey (in plain language)

Flow: FARMER → PHOTO + VOICE → AI ANALYSIS → SPOKEN ANSWER → ACTION

1. Farmer opens the app, sees two big buttons: a camera icon and a microphone icon (minimal text, Urdu labels, icons for those who can't read).
2. They take a photo of the affected leaf/plant and press-and-hold the mic to ask their question in Urdu, e.g., "یہ پتے کیوں بو رہے ہیں؟" ("Why are these leaves turning yellow?").
3. On submit: the photo goes to a vision-capable Qwen model; the voice note is transcribed to Urdu text.
4. The vision model identifies likely visual symptoms (e.g., "yellowing with brown spotting pattern, consistent with early blight").
5. A language model combines the visual finding + the farmer's spoken question + a small local knowledge base (common regional crop issues, low-cost treatments available in local markets) to draft an answer.
6. The model explicitly states a confidence level and, if low, recommends contacting a human expert instead of guessing.
7. The answer is converted to spoken Urdu audio and played back, with a short text summary shown in large Urdu script underneath.
8. The farmer can ask a follow-up by voice; the app keeps the conversation and photo in context for that session.
9. Action: the farmer sees a simple next-step card (e.g., "کم لاگت علاج" — low-cost treatment options) they can act on immediately.

Scenario 1 — Tomato leaf yellowing (medium confidence). Output: "یہ زیادہ تر ابتدائی (70% یقین)۔ متاثرہ پتے ہٹا دیں اور تانبے پر مبنی سپرے استعمال کریں جو جھلساؤ بیماری لگتی ہے۔" ("This looks mostly like early blight (70% confidence). Remove affected leaves and use a copper-based spray available at your nearby shop. If there's no improvement in 3 days, contact an expert.")

Scenario 2 — Unclear/ambiguous photo (low confidence). Output: "مجھے پورا یقین نہیں ہے — علامات ایک سے زیادہ مسئلوں سے ملتی ہیں۔ براہ کرم قریبی زرعی ماہر سے تصدیق کروائیں۔ فی الحال، متاثرہ حصے کو الگ رکھیں۔" ("I'm not fully confident — the symptoms match more than one issue. Please confirm with a nearby agriculture expert. For now, isolate the affected area.")

Scenario 3 — Pest damage, high confidence. Output: "یہ سنڈی کا نقصان لگتا ہے (85% یقین)۔" ("This looks like caterpillar/pest damage (85% confidence). Use neem oil spray in the evening — it's cheap and locally available.")

STEP 6 — The AI, Explained From Zero

- **Machine Learning / Deep Learning:** The umbrella terms for "software that learns patterns from data instead of being explicitly programmed." You don't need to train one from scratch — you're using pre-trained models via API.
- **Generative AI / LLM (Large Language Model):** A model trained on huge amounts of text that can understand and generate natural language. **Why we need it:** to turn a raw diagnosis + question into a natural, farmer-friendly Urdu explanation. **In our project:** Qwen-Plus/Qwen-Max generates the advisory text. **Implementation:** one API call with a well-written prompt (see Step 15).
- **Computer Vision:** Lets a model "look at" an image and describe/classify what's in it. **Why:** to analyze the crop photo. **In our project:** Qwen's vision-capable model (multimodal input) identifies visible symptoms. **Implementation:** send the photo + a instruction prompt to the vision endpoint.
- **NLP (Natural Language Processing):** The broader field of computers understanding language; the LLM above is doing this.
- **Speech AI (ASR + TTS):** ASR ("Automatic Speech Recognition") turns voice into text; TTS ("Text-to-Speech") turns text into voice. **Why:** our users prefer speaking/listening over typing/reading. **In our project:** Qwen's audio models handle both directions. **Implementation:** send recorded audio to the ASR endpoint, send the generated text response to the TTS endpoint.
- **RAG (Retrieval-Augmented Generation):** Instead of relying only on the model's built-in knowledge, you give it a small, curated set of local documents (e.g., a short list of common regional crop issues and affordable treatments) to look up and ground its answer in — reduces made-up answers. **In our project:** a small local knowledge file the model references before answering.
- **Agents:** Software that can take multiple automated steps/decisions on its own. **We do not need a complex agent** for the MVP — a simple linear pipeline (photo → vision → text → speech) is enough and far more reliable to demo.
- **Classification / Prediction:** The vision model is effectively doing classification (which category of symptom this is).
- **Embeddings / Vector database:** Only needed if your knowledge base grows large; for a small hand-curated list of ~30–50 common crop issues, you can skip this and just include it directly in the prompt — simpler and less likely to break during the demo.

What we are NOT using and why: No custom-trained neural network, no vector database, no complex multi-agent framework — all unnecessary complexity for a 5-person beginner team on a hackathon timeline, and each one adds a new point of failure during judging.

STEP 7 — Technical Architecture

```
Farmer (mobile browser)
  ↓
Frontend — simple mobile-first web app (HTML/CSS/JS or lightweight React)
  ↓
Backend API — Python (FastAPI)
  ↓
AI Layer — Alibaba Cloud Model Studio (Qwen: vision, text, ASR, TTS)
  ↓
Small local knowledge file (curated crop-issue + treatment list, loaded into
the prompt)
  ↓
Storage — Alibaba Cloud OSS (photos/audio) + lightweight database for session
history
  ↓
Response → spoken Urdu answer + text summary → back to Farmer
```

Frontend: Plain HTML/CSS/JS (or a minimal React app) — easiest to demo reliably on any judge's laptop/phone, and easiest for a beginner frontend person to fully own and explain.

Skip Flutter/native mobile — too much setup risk for the timeline.

Backend: Python + FastAPI — simple, fast to write, huge beginner-friendly documentation, natural fit for calling the Alibaba Cloud API.

AI: Alibaba Cloud Model Studio (Qwen family) for vision analysis, Urdu-capable text generation, speech-to-text, and text-to-speech, all through one consistent, OpenAI-compatible API: <cite index="7-1,7-2">Model Studio is a one-stop model service platform providing the full Qwen series and mainstream third-party LLMs through official Qwen and OpenAI-compatible APIs, with multimodal coverage across text, image, audio, and video generation.</cite>

Database: A lightweight one (SQLite for local dev, or Alibaba Cloud's managed database if time permits) to store farmer session history (photo, question, answer) for the demo's "context memory" feature.

Storage: Alibaba Cloud OSS (Object Storage Service) for uploaded photos/audio clips.

Deployment: Deploy the backend as a small containerized service; for the hackathon's compliance requirement, keep a Docker setup ready so you can point it at Alibaba Cloud ECS, while developing locally first — this mirrors how other teams have handled the same requirement in similar Qwen/Alibaba hackathons. <cite index="26-1">Eligibility for Alibaba Cloud's free trial requires a registered account with verified phone number and payment method, no overdue payments, and no prior orders for the product you're trying.</cite>

Alibaba Cloud services and why:

- **Model Studio (Qwen APIs)** — vision, text generation, ASR, TTS — the core AI engine.
- **OSS (Object Storage Service)** — stores farmer-submitted photos/audio.
- **ECS (Elastic Compute Service)** — hosts the backend for the "deployed on Alibaba Cloud" proof most Alibaba/Qwen hackathons require.

STEP 8 — Alibaba Cloud Integration Detail

Service	What it does	Why we need it	How to get it	Where it's used
Model Studio (Qwen-VL / Qwen-Plus / Qwen-Max)	Multimodal text + vision generation	Core diagnosis + advisory text	Create an Alibaba Cloud account → activate Model Studio → generate an API key	Backend calls this for every diagnosis
Qwen ASR/TTS (audio models)	Speech-to-text and text-to-speech	Voice-first Urdu interface	Same API key/workspace	Voice input transcription + spoken answer generation
OSS	Object storage	Store photo/audio files	Alibaba Cloud console → OSS bucket	Photo/audio upload target
ECS	Virtual server hosting	Deploy backend for judges to test live	Alibaba Cloud console → ECS instance	Hosts the FastAPI backend

How to create/access: Sign up for an Alibaba Cloud account, activate Model Studio, accept the terms of service, then generate an API key from the API Key page: <cite index="9-1,9-2">to call the Qwen API you create an Alibaba Cloud account, activate Model Studio and accept its terms, then go to the API Key page and create an API key, after which you can call models through OpenAI-compatible or DashScope endpoints.</cite>

Model choice guidance: Don't default to the biggest model. <cite index="7-1">Qwen-Max is the highest-performing model suited for complex, multi-step tasks; Qwen-Plus balances performance, speed, and cost for most scenarios; Qwen-Flash is low-cost and low-latency for simple tasks needing fast responses.</cite> For a live demo, **Qwen-Plus is the safe default** — fast enough for a smooth demo, capable enough for good Urdu output, and cheaper to run through many rehearsals.

Regions: <cite index="7-1,7-2">Model Studio is available in Singapore, US (Virginia), China (Beijing), China (Hong Kong), Japan (Tokyo), and Germany (Frankfurt), and you should pick a nearby region to reduce latency, since regions differ in endpoints, API keys, supported models, and pricing.</cite> For a Pakistan-based team, **Singapore** is the best regional choice for lowest latency.

Cost/trial note: Free trial access requires <cite index="26-1">a registered Alibaba Cloud account with verified phone number and payment method, and no overdue payments or prior orders for that specific product.</cite> Set this up on Day 1, not the night before the deadline — it can involve a small refundable verification charge.

Data privacy note: <cite index="7-1">Alibaba Cloud states it will not use API data for model training, and encrypts data transmitted while building or training applications.</cite> Still — never upload real farmers' personal identifying data during development; use your own test photos.

STEP 9 — Dataset

Dataset	Source	Format	Language	License	Use
PlantVillage dataset	Kaggle (public)	Images (jpg)	N/A (visual)	Open/research use	A handful of sample images for demo/testing the vision prompt, not for training
Self-collected photos	Your own phones — photograph houseplants, vegetable leaves, or use royalty-free stock photos of common crop diseases	jpg	N/A	Your own	Primary demo dataset — 15–20 curated photos covering 5–6 common issue types
Curated crop-issue knowledge list	Compiled manually from agriculture extension publications, PARC (Pakistan Agricultural Research Council) public materials, and general agronomy references	Plain text/JSON	Urdu + English	Your own compiled summary (cite sources, don't copy text verbatim)	Small "grounding" file fed into the LLM prompt (the RAG-lite step in Step 6)

No perfect ready-made dataset exists specifically for "Pakistani-crop + Urdu-advisory" — and that's fine. Build a small (~20–30 entry) hand-curated knowledge file of the most common regional crop issues and affordable local treatments; this is realistic for a beginner team and is also your demonstrable "domain expertise" artifact for judges.

Privacy: Since you are not collecting real farmer data for the hackathon, there's no personal data risk. If this becomes a real product later, add: no photo/audio stored beyond the session unless the user opts in, and a clear on-screen disclaimer (see Step 19).

STEP 10 — Do We Train a Model?

Answer: No. Use existing APIs (Option C), with a small RAG-lite grounding step (Option E-light).

Why: training or fine-tuning your own vision/language model requires labeled data, GPU time, and ML expertise none of you have yet — and it adds massive risk for near-zero visible benefit in a demo. Alibaba Cloud's Qwen models already do strong multimodal understanding and Urdu generation out of the box. Your innovation is in the **product design** (voice-first, confidence-transparent, locally-grounded), not in training a novel model. This is the simplest path that still produces an honestly impressive prototype.

STEP 11 — Complete Tech Stack

Component	Technology	Why
Frontend	HTML/CSS/JS (or minimal React)	Fast to build, easy for a beginner to own fully, reliable for live demo
Backend	Python + FastAPI	Simple, fast, great docs, easy async calls to Qwen API
AI Model	Qwen-VL (vision) + Qwen-Plus (text)	Multimodal, Urdu-capable, one consistent API
AI API	Alibaba Cloud Model Studio (DashScope / OpenAI-compatible endpoint)	Required integration, well-documented
Database	SQLite (dev) → Alibaba Cloud managed DB (if time allows)	Session/history storage, minimal setup
Storage	Alibaba Cloud OSS	Photo/audio storage
Cloud	Alibaba Cloud ECS	Backend hosting for judge-testable deployment
Deployment	Docker container on ECS (or a simple process manager)	Reliable, reproducible, satisfies "proof of Alibaba Cloud deployment"
Authentication	Simple session token or skip entirely for MVP demo	Not core to the value proposition; don't burn time here
Version Control	Git + GitHub	Team collaboration, required for most hackathon submissions
Testing	Manual test-case checklist (Step 18) + basic Python (pytest) for backend endpoints	Enough rigor without slowing the team down

STEP 12 — Skills Required (5 Beginners)

Must learn:

- Basic Python (functions, JSON, API calls)
- Git/GitHub basics (clone, commit, push, pull request, resolve simple conflicts)
- What an API is and how to call one with an API key
- Basic HTML/CSS/JS
- How to write a good prompt for an LLM (prompt engineering basics)

Good to know:

- FastAPI basics (can be learned in a day by 1-2 members and taught to others)
- Basic UI/UX principles for low-literacy users (icons > text, big touch targets)
- How image/audio files are uploaded and handled in a web app

Not necessary:

- Training/fine-tuning ML models
- Deep learning math/theory
- Vector databases / embeddings (skip for MVP)
- Complex agent frameworks
- Advanced cloud architecture (Kubernetes, multi-region, etc.)

STEP 13 — Team Roles (5 Members)

Member 1 — AI/Prompting Lead

Responsibilities: Own the Qwen API integration (vision, text, ASR, TTS calls) and prompt design. **Skills to learn:** API basics, prompt engineering, JSON handling. **Tasks:** Write and test the vision-analysis prompt, the advisory-generation prompt, the confidence-scoring logic, and the small local knowledge file. **Deliverables:** Working Python functions for `analyze_photo()`, `generate_advice()`, `speech_to_text()`, `text_to_speech()`. **What they explain to judges:** How the AI pipeline works end-to-end and why confidence-transparency matters.

Member 2 — Backend/API Lead

Responsibilities: FastAPI server, routing, connecting frontend to AI functions, OSS/database wiring. **Skills to learn:** FastAPI, REST endpoints, file uploads. **Tasks:** Build `/diagnose` and `/ask-followup` endpoints; connect to OSS for storage; connect to Member 1's AI functions. **Deliverables:** A working backend that frontend can call. **What they explain to judges:** System architecture and how data flows from user input to AI response.

Member 3 — Frontend/UX Lead

Responsibilities: The mobile-first web UI — camera capture, mic recording, playing back audio responses, low-literacy-friendly design. **Skills to learn:** HTML/CSS/JS, basic UI/UX for accessibility. **Tasks:** Build the two-button home screen, recording/upload flow, response display (audio player + large Urdu text). **Deliverables:** A working, demo-ready UI. **What they explain to judges:** Why the UI is designed the way it is for the target user (low literacy, voice-first).

Member 4 — Data/Research Lead

Responsibilities: Curate the crop-issue knowledge file, source demo photos, validate real-world accuracy of the sample scenarios, write the problem-research narrative. **Skills to learn:** Basic research methodology, JSON/text data formatting. **Tasks:** Compile ~20-30 common crop issues + affordable local treatments; collect/curate demo photo set; write the "who is suffering" research summary for the pitch. **Deliverables:** The knowledge file, the demo photo set, the problem-statement one-pager. **What they explain to judges:** The evidence behind the problem and why the specific scenarios chosen are realistic.

Member 5 — Cloud/Integration/Testing Lead

Responsibilities: Alibaba Cloud account setup (Model Studio, OSS, ECS), deployment, end-to-end testing, demo rehearsal logistics. **Skills to learn:** Alibaba Cloud console basics, Docker basics, test-case writing. **Tasks:** Set up and manage API keys/credits, deploy backend to ECS, run the test-case checklist (Step 18), prepare the offline-safe demo fallback (Step 20). **Deliverables:** A live, judge-testable deployed URL + a recorded backup demo video. **What they explain to judges:** The Alibaba Cloud architecture and how it's actually deployed.

Collaboration rule: Every member reviews and can explain every other part at a basic level — do a 15-minute "explain your part to the team" session daily, and pair up (AI+Backend, Frontend+UX research, Cloud+everyone) so no one becomes a single point of failure.

STEP 14 — 5-Day Learning Crash Course (Learn by Building)

Day 1 — Foundations

- Learn: What an API is, how to call one with `requests`/`fetch`; Git/GitHub basics.
- Why: Everything else depends on this.
- Resource: freeCodeCamp "APIs for Beginners" (YouTube, free); GitHub's own "Hello World" guide.
- Time: ~3 hours.
- Mini-task: Each member makes a GitHub account, clones a test repo, makes one commit.

Day 2 — Your first Qwen API call

- Learn: Alibaba Cloud account setup, Model Studio activation, first API call.
- Why: This is your entire AI engine.
- Resource: Alibaba Cloud Model Studio "Get Started" docs (official).
- Time: ~3 hours.
- Mini-task: Every member successfully sends one text prompt to Qwen and prints the response.

Day 3 — Prompt engineering + vision calls

- Learn: How to write clear, structured prompts; how to send an image to a multimodal model.
- Why: This is the core "diagnosis" logic.
- Resource: Anthropic/OpenAI/Alibaba's public prompt-engineering guides (concepts transfer across providers).
- Time: ~4 hours.
- Mini-task: Send one test crop photo to Qwen-VL and get back a description.

Day 4 — Backend + frontend basics

- Learn: FastAPI "hello world" endpoint; basic HTML form for file upload.
- Why: You need a way for users to reach the AI.
- Resource: FastAPI official tutorial (first 3 sections only).
- Time: ~4 hours.
- Mini-task: Build one endpoint that accepts a photo and returns dummy text.

Day 5 — Speech (ASR/TTS) + connect the pieces

- Learn: How to send audio to a speech-to-text endpoint and get audio back from text-to-speech.
- Why: This is your voice-first differentiator.
- Resource: Alibaba Cloud Model Studio audio API docs.
- Time: ~4 hours.
- Mini-task: Record a 5-second Urdu voice note, transcribe it, and play back a synthesized reply.

By end of Day 5, every member has personally touched every layer of the stack once.

STEP 15 — Development Roadmap

Phase	Who	Task	Output	Tools	Time	Depends on
1. Problem validation	All	Confirm problem via research/interviews if possible	Problem one-pager	Notes	0.5 day	—
2. Research	Member 4	Curate crop-issue knowledge file, demo photos	Knowledge file + photo set	Docs, Kaggle	1 day	Phase 1
3. Cloud setup	Member 5	Alibaba Cloud account, Model Studio, API key, OSS bucket	Working credentials	Alibaba Cloud console	0.5 day	—
4. AI pipeline	Member 1	Prompt design + test calls (vision, text, ASR, TTS)	Working AI functions	Python	1.5 days	Phase 3
5. Backend	Member 2	FastAPI endpoints, connect AI functions, OSS wiring	Working API	FastAPI	1.5 days	Phase 4
6. Frontend	Member 3	UI screens, camera/mic capture, audio playback	Working UI	HTML/CSS/JS	1.5 days	Phase 5 (can start in parallel with mocked data)
7. Integration	Members 2+3	Connect frontend ↔ backend end-to-end	Working full app	—	1 day	Phases 5+6
8. Deployment	Member 5	Deploy backend to ECS, connect frontend	Live URL	Docker, ECS	0.5 day	Phase 7
9. Testing	All	Run test-case checklist, fix bugs	Bug-free demo path	Manual + pytest	1 day	Phase 8
10. Presentation	All	Slides, speaking parts, demo rehearsal	Ready pitch	Slides tool	1 day	Phase 9

Total: roughly 10 working days — compress proportionally to your actual hackathon length.

STEP 16 — GitHub Structure

```
fasaldoc/
├── frontend/          # HTML/CSS/JS UI
├── backend/           # FastAPI app
│   ├── main.py
│   ├── ai/            # AI pipeline functions (Member 1's code)
│   ├── routes/        # API endpoints (Member 2's code)
│   └── storage/       # OSS integration helpers
├── data/              # Curated knowledge file + demo photos
├── docs/              # Problem research, architecture diagram, pitch
├── notes
├── tests/             # pytest test cases
├── README.md
├── requirements.txt
└── .env.example       # placeholder for API keys — never commit real keys
```

Collaboration rules: Each member works on their own branch (`feature/ai-pipeline`, `feature/backend-api`, etc.), opens a pull request when ready, and at least one other member reviews before merging to `main`. Use GitHub Issues to track tasks so work doesn't overlap. Never commit `.env`/API keys — add them to `.gitignore` on Day 1.

STEP 17 — MVP vs Advanced

MUST HAVE (MVP): Photo upload → vision analysis → Urdu text advice → basic voice playback of the answer; one working demo scenario end-to-end.

SHOULD HAVE: Full voice-in (ASR) follow-up questions; confidence-score display; 3 polished demo scenarios.

WOW FEATURES (only after MVP works): Conversation memory across a session; a simple map/list of nearby input shops (even mocked); multi-language toggle (Urdu/Punjabi/English).

DO NOT BUILD: Custom model training, user accounts/login system, payment features, a native mobile app, a large vector database — none of these change the judged outcome and all add failure risk.

STEP 18 — Testing

Test cases:

- Clear photo of a known issue → correct/plausible diagnosis with high confidence.
- Blurry/ambiguous photo → low-confidence response that recommends a human expert (this is the case to rehearse most — it's your differentiator).
- Voice input with background noise → does transcription still work reasonably?
- Empty/no photo submitted → graceful error message, not a crash.
- Non-plant photo submitted → the model should say it can't help, not hallucinate a diagnosis.

Evaluation approach: Since you don't have a large labeled test set, use a simple rubric — for each of your ~15–20 demo photos, does the model's answer *make agronomic sense* to a knowledgeable reviewer (check against your Step 9 knowledge file)? Track a simple "reasonable / not reasonable" pass rate.

Communicating uncertainty: Never present the AI's answer as a certain fact. Always show a confidence indicator and, below a set threshold (e.g., under ~60%), replace the specific diagnosis with a "please consult an expert" message. This is both good practice and a strong judge-facing talking point.

STEP 19 — Security, Privacy, Ethics

- **Privacy:** Don't collect real farmer identity data during the hackathon. If this became a real product, store only what's needed (photo/audio per session), let users delete their data, and avoid tying data to a real name/phone number unless necessary.
 - **Data security:** Use HTTPS for all API calls; never expose your Alibaba Cloud API key in frontend code — keep it server-side only.
 - **Bias/limitations:** The model is trained mostly on general (often non-Pakistan-specific) agricultural data — it can misdiagnose regional or unusual conditions. State this openly.
 - **Hallucination risk:** The model can produce a confident-sounding wrong answer. The confidence-threshold fallback (Step 18) is your main mitigation.
 - **Misuse:** Could be misused to justify unnecessary pesticide purchases if trusted blindly — mitigate by always suggesting the *lowest-cost, least-toxic* option first and recommending expert consultation for anything serious.
 - **Suggested in-app disclaimer:** "معاون ہے اور ماہر زرعی مشورے کا متبادل نہیں۔ یہ ایک AI۔" ("This is an AI assistant and not a substitute for expert agricultural advice. For serious situations, please consult a local agriculture expert.")
-

STEP 20 — 3-5 Minute Live Demo Script

- **0:00 — Problem:** "Every season, small farmers in Pakistan lose money because they can't quickly and reliably tell what's wrong with their crops, and the person they usually ask — the pesticide dealer — profits from overselling."
- **0:30 — Introduce solution:** "FasalDoc lets a farmer just take a photo and ask a question out loud, in Urdu — no reading, no typing."
- **1:00 — User opens app:** Show the two-button home screen live.
- **1:30 — User provides input:** Take a real photo of a pre-prepared leaf sample; record a short Urdu voice question.
- **2:00 — AI processes it:** Show the loading state; narrate briefly what's happening behind the scenes.
- **2:30 — Result appears:** Play the spoken Urdu answer; show the confidence score and the text summary on screen.
- **3:00 — Explain the AI:** 20-second, plain-language explanation of the vision + language + speech pipeline.
- **3:30 — Alibaba Cloud:** Show the architecture diagram slide; mention Model Studio, OSS, ECS specifically.
- **4:00 — Impact:** State the scale of the problem and who benefits.
- **4:30 — Future potential:** Mention nearby-shop integration, multi-language expansion, extension-officer partnership.

Reliability tip: Pre-record a backup video of this exact flow in case live Wi-Fi/API calls fail during judging — this is standard practice and shows foresight, not weakness.

STEP 21 — Slide Structure

#	Title	Content	Visual	Presenter
1	Title	FasalDoc + tagline + team names	Logo/mockup	Member 3
2	The Problem	Who suffers, why current solutions fail	Simple stat/photo	Member 4
3	The Solution	One-sentence pitch + 3-scenario overview	Screenshot	Member 3
4	Live Demo	(switch to live app)	—	Member 2
5	How the AI Works	Vision → language → speech pipeline, plain language	Simple flow diagram	Member 1
6	Architecture	Full technical architecture	Architecture diagram	Member 2
7	Alibaba Cloud Integration	Which services, why	Alibaba Cloud logo diagram	Member 5
8	Innovation	What's different vs. existing tools	Comparison table	Member 4
9	Impact & Next Steps	Social impact + roadmap	Simple growth chart	Member 4
10	Team & Thanks	Roles, contact	Team photo	All

Keep each slide to under ~25 words of text; let the presenter's voice carry the detail.

STEP 22 — Speaking Division (5 Members)

Member 1 (AI): *"I built the AI pipeline that looks at the crop photo, listens to the farmer's question, and generates an honest answer — including telling the farmer when the AI isn't sure." — Likely question: "How accurate is your vision model?" — Answer: "We tested against ~20 curated real-world-style cases and it gave agronomically reasonable answers in the clear majority; for ambiguous cases, it correctly defers to a human expert instead of guessing — that's by design."*

Member 2 (Backend): *"I connected the AI to a working API that the app calls in real time, and handled the file/audio upload flow." — Q: "What happens if the AI API is slow or down?" — A: "We show a friendly loading state, have a timeout with a retry, and a pre-recorded backup demo as a fallback."*

Member 3 (Frontend): *"I designed the interface specifically for someone who may not read comfortably — big icons, voice-first, minimal text." — Q: "Why not just build a normal app with menus?" — A: "Our target users are more comfortable speaking than typing or navigating menus; every extra screen or text field is a barrier we chose to remove."*

Member 4 (Research): *"I researched who actually suffers from this problem and why current solutions — mainly pesticide dealers — fail them." — Q: "Where did your data/knowledge base come from?" — A: "A hand-curated list of common regional crop issues and affordable treatments compiled from public agriculture references, which we openly acknowledge is a starting point, not exhaustive."*

Member 5 (Cloud): *"I set up and deployed our whole backend on Alibaba Cloud — Model Studio for the AI, OSS for storage, ECS for hosting." — Q: "Why Alibaba Cloud specifically?" — A: "Model Studio gave us one consistent API for vision, language, and speech — which is exactly the multimodal pipeline this problem needed, without stitching together three different vendors."*

STEP 23 — 30 Judge Q&A (Honest Answers)

1. *Why does this problem matter?* — Crop loss and pesticide misuse directly hit farmer income and food security; small identification errors compound at national scale.
2. *Why AI?* — Visual symptom pattern-matching and natural Urdu conversation at this speed aren't practical with rule-based software.
3. *Why not a normal (non-AI) app?* — A static FAQ can't interpret a specific photo or answer an open-ended spoken question.
4. *Why is your solution different?* — Voice-first Urdu UX + confidence-transparent answers + locally-grounded advice, combined, for a farmer specifically.
5. *Why should users trust it?* — It says when it's unsure and defers to a human expert rather than guessing.
6. *Where did your data come from?* — Public plant-image datasets for demo images, hand-curated agronomy references for the knowledge base — clearly disclosed, not a large proprietary dataset.
7. *How accurate is it?* — We honestly report it as "reasonable on our tested cases, not clinically validated" — appropriate for a prototype, not a certified diagnostic tool.
8. *What happens when the AI is wrong?* — Confidence threshold triggers an "ask an expert" fallback instead of a false-confidence answer.
9. *Why did you choose this model?* — Qwen-Plus balances speed and quality for a smooth live demo while supporting multimodal input natively.
10. *Why Alibaba Cloud?* — One platform covers vision, language, and speech, matching our multimodal need exactly.
11. *What happens at scale?* — More usage means more API cost, but no architecture change; the main future work is expanding the knowledge base regionally.
12. *What's your business model?* — Freemium: free basic diagnosis, paid tier for input-shop partnerships/affiliate advisory for larger farms or agri-businesses.
13. *How could this become a real startup?* — Partner with agriculture extension programs or telecom-based rural services (many Urdu voice-AI vendors already sell into telecom channels).
14. *What's your biggest limitation?* — Small hand-curated knowledge base and unvalidated accuracy — it's a prototype, not a certified tool yet.
15. *What would you build next?* — Expand the knowledge base with real extension-officer input, add regional dialect support, validate accuracy with real farmer testing.
16. *Is this safe to use without a human in the loop?* — No — it's explicitly designed to defer to humans for anything uncertain or serious.
17. *How do you handle bad internet in rural areas?* — Lightweight audio/image compression before upload; a future offline-first mode is a known next step.
18. *What if the photo quality is poor?* — The model is prompted to lower its confidence rather than force a guess.
19. *How is this different from a Google Lens plant ID feature?* — Plant ID tools identify species; we diagnose disease/pest issues and give locally-relevant next steps in Urdu, by voice.
20. *Is Urdu really necessary — why not English?* — Our target users are more comfortable in spoken Urdu than written English; this removes the single biggest usability barrier.
21. *How did you validate the problem is real?* — Existing commercial Urdu voice-AI vendors already market crop-disease-alert scenarios, confirming market recognition of the need.
22. *What's your competitive moat?* — Not a moat of secret technology — it's user-experience design for an underserved user segment, which is defensible through iteration and trust.
23. *How many users have you tested with?* — Honestly: none yet in the field for this hackathon prototype; that's the clear next validation step.
24. *What is the cost per query?* — Roughly the cost of one Qwen-Plus multimodal call plus one ASR/TTS call — cheap enough for a freemium model at small scale.

25. *Could this replace agriculture extension officers?* — No — it's designed to extend their reach, not replace expert judgment for serious cases.
 26. *What about farmers without smartphones?* — Out of scope for this prototype; a future version could use basic USSD/IVR voice calls instead of a smartphone app.
 27. *How do you prevent harmful pesticide recommendations?* — We prioritize lowest-toxicity/lowest-cost options and always recommend expert confirmation for anything above a moderate confidence threshold.
 28. *What's the environmental impact?* — Positive: reducing unnecessary/incorrect pesticide use is itself an environmental benefit.
 29. *Why should this win over a healthcare/education AI project?* — It targets a domain with less competition and a more concrete, demoable real-world impact loop.
 30. *What's the one thing you'd want us to remember?* — "An AI that admits when it doesn't know is more trustworthy than one that always sounds confident."
-

STEP 24 — Business & Scalability

- **Target market:** Small/mid-scale farmers across Pakistan (and later, similar Urdu/Hindi-speaking farming regions in South Asia).
 - **Possible customers/partners:** Agriculture extension programs, telecom operators (who already sell Urdu voice-AI customer service — a natural distribution partner), agri-input retailers, NGOs.
 - **Revenue model:** Freemium core diagnosis; paid B2B tier for agri-retailers/telecoms wanting to white-label or bundle the tool.
 - **Scalability:** API-based architecture scales horizontally; main future cost driver is per-query AI API cost, offset by a freemium/subsidized model.
 - **Approximate future costs:** Dominated by AI API usage costs at scale, plus modest cloud storage/hosting — no heavy infrastructure required since there's no self-hosted model.
 - **Path to a real business:** Start as an NGO/extension-partnership pilot, gather real accuracy/usage data, then pursue telecom or agri-retail partnerships for distribution.
-

STEP 25 — Competition Strategy

Self-scored estimate (be honest with yourselves, then let judges decide): Innovation 7 | Real-world impact 9 | Technical complexity 6 | AI usage 8 | Alibaba Cloud usage 8 | Feasibility 8 | UX 8 | Scalability 7 | Presentation potential 8 | **Overall winning potential 7.5/10**

What could make this project win: A flawless, fast, emotionally resonant live demo; genuine honesty about AI limitations (rare among hackathon teams); every team member clearly owning and explaining a real technical part; a strong, specific Alibaba Cloud integration story instead of a vague one.

What could cause it to lose: Live demo failure with no backup plan; overclaiming medical/agronomic accuracy; a rushed, generic pitch that doesn't differentiate from "just another chatbot"; uneven team participation where only one person can answer technical questions.

STEP 26 — Backup Plan

1. **API failure during demo.** *Why it happens:* network issues, rate limits, live Wi-Fi problems. *Prevention:* test the exact demo flow multiple times beforehand; keep API calls minimal during the live demo. *Backup:* a pre-recorded video of the full working flow, ready to play instantly.
2. **Model gives a nonsensical/wrong answer live.** *Why:* AI is probabilistic, not perfect. *Prevention:* use pre-tested photos/questions you know produce good results for the live demo. *Backup:* have 2-3 alternate tested photo/question pairs ready if the first attempt goes badly.

3. **Dataset/knowledge gaps make an answer look unconvincing.** *Why:* small hand-curated dataset. *Prevention:* Member 4 validates every demo scenario in advance against the knowledge file. *Backup:* be transparent in the pitch that the knowledge base is a starting point, not exhaustive — judges respect honesty over overclaiming.
 4. **Deployment/ECS issues close to deadline.** *Why:* new cloud account setup issues, time pressure. *Prevention:* set up Alibaba Cloud access on Day 1, not the last day. *Backup:* have a working local/alternative-hosted version with a Dockerfile ready as proof of Alibaba Cloud integration in code, even if live ECS hosting has issues — this mirrors how other participants in similar Qwen hackathons have handled the same deployment requirement.
 5. **Running out of time before the deadline.** *Why:* scope creep, beginner team. *Prevention:* strictly follow the MVP-first plan in Step 17; treat "should have" and "wow" features as optional. *Backup:* a working MVP with one polished demo scenario beats an ambitious, broken feature set.
-

STEP 27 — Final Execution Checklist

Research ☐ Problem validated with evidence/sources ☐ Existing solutions researched and documented

Learning ☐ All 5 members completed the Day 1-5 crash course ☐ Every member can explain the full pipeline at a basic level

Data ☐ Knowledge file (~20-30 entries) compiled ☐ Demo photo set (15-20 images) collected

Coding ☐ Backend endpoints working ☐ Frontend UI working end-to-end ☐ Frontend + backend integrated

AI ☐ Vision analysis tested and reliable on demo photos ☐ Text advisory generation tested in Urdu ☐ ASR/TTS tested and reliable

Alibaba Cloud ☐ Account + Model Studio activated, API key working ☐ OSS bucket configured ☐ ECS deployment live and tested

Testing ☐ All test cases from Step 18 run ☐ Low-confidence fallback verified working

Deployment ☐ Live URL tested from a different device/network ☐ Backup demo video recorded

Presentation ☐ Slides finalized ☐ Each member's speaking part rehearsed

Demo ☐ Live demo rehearsed at least 3 times end-to-end ☐ Backup plan ready if live demo fails

GitHub ☐ Repo structured per Step 16 ☐ README explains setup + team roles ☐ No API keys committed

Final Submission ☐ All hackathon submission requirements double-checked (deployment proof, team info, etc.) ☐ Submitted before deadline with buffer time

Good luck — the strongest thing about this project isn't the AI model, it's that you built something an actual farmer could use and trust, and you can prove you understand every part of it.